

Data Engineer Interview Prep — Doc

2

Databricks, Delta Lake, AWS (S3/Glue/Lambda), SQL, data modeling, CI/CD

Q&A + prep focus per area

Tailored to a JD emphasizing: **Databricks (Spark, Delta Lake), Python ETL, AWS (S3/Glue/Lambda), SQL tuning, data lake architectures, ELT/ETL, orchestration, dimensional & normalized data models, and engineering standards (CI/CD, version control, code quality, security, documentation)**. Each section: **prep focus + Q&A**.

SECTION 1 — Databricks (Spark + Delta Lake)

Prep focus: Databricks concepts (workspace, clusters, notebooks, jobs, Unity Catalog), Delta Lake features (ACID, MERGE, time travel, OPTIMIZE/Z-ORDER), and the medallion pattern.

Q: What is Databricks and why use it? A managed **lakehouse** platform built on **Apache Spark**. It gives collaborative notebooks, managed Spark clusters, jobs/workflows for scheduling, **Delta Lake** for reliable tables, and **Unity Catalog** for governance — so you run big data ETL, SQL, and ML in one place without managing infrastructure.

Q: What is Delta Lake and what problems does it solve? An open **table format** on top of Parquet in the lake that adds:

- **ACID transactions** (reliable concurrent reads/writes — no partial/corrupt data),
- **MERGE /upserts and deletes** (updates on a lake that's otherwise append-only),
- **Time Travel** (query/rollback to a previous version),
- **Schema enforcement + evolution**,
- Performance via **OPTIMIZE** (compaction) and **Z-ORDER** (data skipping). It turns a messy Parquet lake into a reliable, warehouse-like store.

Q: Explain the medallion architecture. Three Delta layers: **Bronze** (raw ingested), **Silver** (cleaned/validated/joined), **Gold** (business-level aggregates/models for BI). Each stage is reproducible; you reprocess from Bronze.

Q: How does **MERGE (upsert) work in Delta?**

```
MERGE INTO gold.customers t USING updates s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

It applies inserts/updates (and deletes) atomically — the core of incremental loads and **SCD Type 2**.

Q: How do you optimize Delta tables? **OPTIMIZE** to compact small files; **Z-ORDER BY** a frequently-filtered column for data skipping; partition sensibly; **VACUUM** to remove old files; enable **Auto Optimize**. Watch the **small-files problem** from streaming/frequent writes.

Q: What is Unity Catalog? Databricks' centralized **governance** — catalogs/schemas/tables with fine-grained (table/ column/row) access control, lineage, and auditing across workspaces.

Q: Cluster types / job clusters? Interactive clusters for development (notebooks), **job clusters** that spin up per scheduled job (cheaper, isolated). Right-size worker type/count; use autoscaling; use **Photon** for faster SQL.

Q: Databricks Workflows / how do you schedule? Databricks **Workflows (Jobs)** run notebooks/scripts on a schedule or trigger, with dependencies and retries; or orchestrate from **Airflow**. **Delta Live Tables (DLT)** offers declarative, managed pipelines with built-in quality expectations.

SECTION 2 — Spark & Python ETL

Prep focus: Spark execution (lazy, DAG, stages, shuffle), tuning, and clean Python ETL.

Q: How does Spark execute a job? Lazily: transformations build a **DAG**; an **action** triggers it. The **driver** plans, the **cluster manager** gives **executors**, work is split into **stages** at shuffle boundaries and **tasks** (one per partition) run in parallel. Narrow ops (filter/map) stay on the partition; wide ops (join/groupBy) cause a **shuffle**.

Q: How do you tune a slow Spark job? **Broadcast** small tables (avoid big-table shuffle), filter/select early, fix **data skew** (salting, AQE), right-size partitions (`repartition` / `coalesce`), cache reused DataFrames, avoid Python **UDFs** (use built-ins/pandas UDFs), and read `explain()` for `Exchange` /join type.

Q: repartition vs coalesce? `repartition(n)` full shuffle (up or down); `coalesce(n)` no shuffle, only reduces — use before writing to cut small files.

Q: How do you write clean, reliable Python ETL? Modular functions, config-driven (no hard-coding), parameterized, **idempotent**, logging + error handling, type hints, unit tests (`pytest`), and reusable transformation modules; use `boto3` for AWS, generators/chunking for large data.

SECTION 3 — AWS (S3, Glue, Lambda)

Q: How do S3, Glue, and Lambda fit a data workload? **S3** = the data lake (raw→curated, Parquet, partitioned). **Glue** = serverless Spark ETL + Data Catalog + crawlers. **Lambda** = lightweight, event-driven glue (e.g., trigger a job when a file lands, small transforms, notifications). Together: file lands in S3 → **EventBridge/S3 event** → Lambda triggers a Glue/Databricks job → catalog → serve.

Q: When Lambda vs Glue vs Databricks? Lambda for short (<15 min), lightweight, event-driven tasks. Glue for serverless Spark batch ETL + cataloging. Databricks for heavy/interactive Spark, Delta, and ML.

Q: How do you trigger pipelines on new data? **S3 event notifications / EventBridge** → Lambda or Step Functions → the processing job. Event- driven beats polling.

SECTION 4 — SQL (wrangling, transformation, performance tuning)

Q: How do you tune a slow SQL query? Read the **execution plan** (`EXPLAIN`), index/filter/join columns, avoid `SELECT *`, filter early, reduce scanned data (partitions/Parquet), refresh **stats**, avoid functions on indexed columns, and prefer set-based over row-by-row.

Q: Window functions example (top-N per group / running total)?

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn FROM emp;      -- rank
SELECT id, SUM(amt) OVER (PARTITION BY cust ORDER BY dt) running FROM orders;      --
running total
```

Q: How do you deduplicate keeping the latest record? `ROW_NUMBER() OVER (PARTITION BY key ORDER BY updated_at DESC)` and keep `rn = 1` — the everyday CDC/upsert pattern.

Q: Common data-wrangling SQL? Joins, `GROUP BY`/aggregations, `CASE` logic, `COALESCE` / `NULLIF` for nulls, CTEs for readability, `PIVOT` /conditional aggregation, date functions.

SECTION 5 — Data Modeling (dimensional & normalized)

Prep focus: this JD explicitly wants both — know star schema vs 3NF and SCD.

Q: Dimensional vs normalized modeling — difference and when?

- **Normalized (3NF):** minimal redundancy, many related tables — great for **OLTP/write-heavy** systems and clean source-of-truth storage.
- **Dimensional (star schema):** **fact** tables (measurable events) + **dimension** tables (descriptive context), denormalized for **fast, simple analytics/BI**. Use 3NF for operational/integration layers; dimensional (star) for the analytics/serving layer.

Q: Star schema vs snowflake schema? Star = dimensions denormalized into single tables (fast, simple). Snowflake = dimensions further normalized into sub-tables (less redundancy, more joins). Star is usual for BI.

Q: Fact vs dimension table? **Fact** = the numbers/events (order amount, quantity) + foreign keys. **Dimension** = the who/what/when/where (customer, product, date).

Q: What are Slowly Changing Dimensions (SCD)? How you track changes to dimension attributes over time. **Type 1** = overwrite (no history); **Type 2** = add a new row with effective/expiry dates + current flag (**keeps history** — the common interview answer); Type 3 = keep previous value in a column.

Q: What is a surrogate key and why use it? A system-generated key (integer/hash) for dimension rows instead of the business/natural key — stable, efficient joins, and it supports SCD Type 2 (multiple versions of the same business key).

Q: How do you design a data model for a new use case? Identify the business process → the **grain** (one row = ?) → the **facts** (measures) → the **dimensions** (context) → choose star schema; add SCD where history matters; define keys and conformed dimensions for cross-functional reuse.

SECTION 6 — ELT/ETL, Data Lakes & Orchestration

Q: ELT vs ETL and your preference on a lakehouse? On Databricks/lakehouse I lean **ELT** — land raw (Bronze), transform in-place with Spark/SQL (Silver/Gold), which scales and keeps raw for reprocessing. ETL when I must transform/mask before landing.

Q: How do you orchestrate? Databricks **Workflows** or **Airflow**/Step Functions — DAGs with dependencies, retries, alerting; event triggers on new files; **DLT** for declarative pipelines with quality checks.

Q: Data lake architecture best practices? Medallion zones, Parquet/**Delta**, partition by date, catalog + governance (Unity Catalog/ Lake Formation), compaction, and immutable raw.

SECTION 7 — Engineering Standards (CI/CD, version control, quality, security, docs)

Prep focus: this JD calls it out — show software-engineering maturity.

Q: How do you apply CI/CD to data pipelines? Code in **Git**; PR reviews; CI runs **linting + unit/integration tests** (and dbt/Databricks tests) on each change; deploy via **Terraform/CloudFormation** and Databricks **asset bundles/ Repos** through dev→staging→prod. **Slim CI** rebuilds only changed models.

Q: How do you ensure code quality and standards? Version control + code review, linters/formatters, **unit tests** for transformations, modular reusable code, naming/style conventions, and **data-quality tests** (Great Expectations/DLT expectations/dbt tests) that fail bad data.

Q: How do you handle security in pipelines? Least-privilege IAM roles / Unity Catalog grants, **secrets in a vault** (Databricks Secrets/ Secrets Manager, never in code), encryption (KMS) at rest + TLS in transit, PII masking/column security, and audit via CloudTrail/Unity Catalog lineage.

Q: How do you document your work? README/design docs, inline docstrings, data **lineage** and a data catalog, notebook markdown, and pipeline runbooks — so the team can understand, operate, and extend it.

Q: How do you collaborate with analytics/product/engineering teams? Gather requirements early, agree on data contracts/SLAs, expose well-modeled tables, communicate changes, and provide docs/quality guarantees so teams get **timely, accurate** data.

QUICK CHECKLIST (this JD)

- [] Delta Lake: ACID, **MERGE**, time travel, **OPTIMIZE** /Z-ORDER, medallion.
- [] Spark tuning: broadcast, skew/AQE, repartition/coalesce, avoid UDFs.
- [] S3 + Glue + Lambda event-driven pattern.
- [] SQL tuning + window functions + dedup.
- [] **Dimensional (star) vs 3NF**, fact/dimension, **SCD Type 2**, surrogate keys.
- [] CI/CD for data (Git, tests, Terraform, Databricks Repos/bundles) + secrets/PII security.
- [] 3+ years framed as: architect/build/optimize pipelines + collaborate cross-functionally.

